

Failure (a loop that won't converge, and a cost-cutting pass) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangGraph), **Advanced** (how a real production system handles both problems). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what each deeper level adds.

This exercise has two separate parts — a non-converging generator ↔ critic loop, and a cost-cutting pass on a working 4-agent run. Both hints cover both parts.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Part 1 — the non-converging loop. Build a generator (writes something) and a critic (checks it, and can reject it) that hand off to each other, the same generator → critic → revision shape from Core Concepts. Then deliberately give the critic criteria nothing could ever satisfy — this isn't a bug you're chasing, it's a scenario you're building on purpose. The whole point is watching what happens when a loop that could run forever meets a real, hard limit.

Part 2 — the cost-cutting pass. Take a working multi-agent run (reuse `supervisor_compare`'s supervisor graph, or Project 4's own 4-agent pipeline if you've started it) and find its single most expensive step. Then try to cut the total cost or time by 30%+ without breaking what the run actually produces.

Things to use:

- A `revision_count` field in state, and a `MAX_REVISIONS` constant — the hard limit.
- A critic with criteria worded so nothing could pass (e.g. "reject unless the draft is under 10 words AND cites 3 sources").
- Your `run_supervisor()` -style report from `sequential_measure` / `supervisor_compare`, reused to profile which stage costs the most.
- One or two concrete cost-cutting levers: a shorter prompt, a cheaper model for one stage, or caching a repeated call.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Part 1. The loop itself is exactly the Generator → Critic → Revision pattern from Core Concepts: a `writer` node produces a draft, a `critic` node either accepts it (`goto=END`) or rejects it and sends it back (`goto="writer"`). What's different here is the critic's own acceptance check — make it genuinely unsatisfiable, and make the limit check happen in the loop itself, not as an afterthought:

- Increment `revision_count` every time the critic rejects, before deciding where to route next.
- Check `revision_count >= MAX_REVISIONS` **before** asking the critic to judge again — if the limit's hit, stop and report, don't run one more doomed round.
- The final report needs to say clearly that the loop was stopped by the limit, not that it succeeded — "couldn't converge after N attempts" is a completely different outcome than a silently truncated success.

Part 2. Profiling means measuring, not guessing — reuse the `StageResult` /report pattern from `sequential_measure` across all of a real run's stages, then look at the numbers to find the actual most expensive one (it's often not the one you'd guess). Common real levers, in order of how much they usually help without hurting correctness:

- A shorter, more focused prompt for the expensive stage (fewer tokens in, often fewer out too).
- A cheaper/smaller model for a stage that doesn't need the most capable model (routing decisions and simple checks are common candidates; final-answer generation usually isn't).
- Caching a call that would otherwise repeat identical work (e.g. the same research query run twice in one session).

Sketch the unsatisfiable critic and your profiling pass before Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what makes a "couldn't converge" report actually useful versus just technically true. A report that says only "gave up after 3 tries" tells whoever reads it that something failed, but nothing about *why* — was the critic's bar genuinely impossible, was the writer stuck repeating the same mistake, or did the two sides disagree about something subjective with no real fix? A production on-call engineer reading this report at 2am needs to be able to tell those apart without re-running the whole thing.

For the cost cut, the harder question isn't "did it get cheaper" — cutting a prompt to one word makes it cheaper and also makes it useless. The real design question is: **how do you prove the cut didn't break correctness, using the same rigor `sequential_measure` and `supervisor_compare` used to prove the cost numbers, not just a vibe check on one example?**

The extra pieces needed:

- Each rejection reason kept in the report (not just the count), so the final "couldn't converge" output shows *what* the critic objected to on every attempt — a reader can see if it was the same objection every time (writer is stuck) or a different one each time (criteria may be contradictory).
- A small, fixed test set (3-5 inputs) run through both the pre-cut and post-cut versions of the pipeline, with outputs compared — not just the token/time totals, so a cut that saves cost but silently degrades quality gets caught before it ships.
- A specific, written percentage for the achieved cut, alongside a specific, written statement of what (if anything) changed in the output quality — "30% cheaper, same 5 test outputs, no observable quality change" is the actual deliverable, not just a smaller number.

Sketch this hardened version yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names both parts and the exact pieces — a revision counter and limit, an unsatisfiable critic, a profiling pass, and concrete cost levers. Intermediate shows exactly where the limit check goes in the loop and which levers to reach for first, in order of how safe they are. Advanced asks what makes each part's output actually trustworthy — a "couldn't converge" report detailed enough to diagnose, not just announce, and a cost cut proven against a real test set rather than assumed safe because the number went down — which is the difference between a report someone can act on and one that just says "it broke."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

Part 1 -- non-converging loop:

```
writer_node: writes a draft, goes to critic
critic_node:
  check draft against impossible criteria (always fails)
  revision_count += 1
  if revision_count >= MAX_REVISIONS: report "couldn't converge", stop
  else: send back to writer_node
```

Part 2 -- cost-cutting pass:

```
run the working pipeline once, print each stage's tokens/seconds
find the most expensive stage
try one cut (shorter prompt / cheaper model / caching) on just that stage
re-run the same test inputs, compare tokens/seconds AND outputs
report: percent saved, and whether outputs still look right
```

Expected output if you run just this (nothing calls the loop yet): nothing — the loop only runs once you invoke the graph with a starting `revision_count` of 0. Add that call to see it actually hit the limit and stop.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# convergence_and_cost_cutting_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

MAX_REVISIONS = 3

class LoopState(TypedDict):
    draft: str
    revision_count: int
    converged: bool
    last_rejection_reason: str

def writer(state: LoopState) -> Command:
    draft = f"[draft attempt {state['revision_count'] + 1}]"
    return Command(update={"draft": draft}, goto="critic")

def critic(state: LoopState) -> Command:
    # impossible criteria on purpose: rejects every draft, always
    passed = False
    reason = "draft does not meet required standard (criteria cannot be satisfied"

    if passed:
        return Command(update={"converged": True}, goto=END)

    new_count = state["revision_count"] + 1
    if new_count >= MAX_REVISIONS:
        return Command(
            update={"revision_count": new_count, "converged": False, "last_rejection_reason": reason},
            goto=END,
        )
    return Command(
        update={"revision_count": new_count, "last_rejection_reason": reason},
        goto="writer",
    )
```

```

builder = StateGraph(LoopState)
builder.add_node("writer", writer)
builder.add_node("critic", critic)
builder.add_edge(START, "writer")
graph = builder.compile()

result = graph.invoke({"draft": "", "revision_count": 0, "converged": False, "last_draft": ""})
if result["converged"]:
    print("Accepted:", result["draft"])
else:
    print(f"Couldn't converge after {result['revision_count']} attempts: {result['draft']}")

```

Expected output:

```

Couldn't converge after 3 attempts: draft does not meet required standard (criteria not met)

```

For the cost-cutting half, reuse `run_supervisor()`'s report shape from `supervisor_compare` across a real pipeline's stages, find the most expensive one by comparing `.tokens`, then try a shorter prompt on it and re-measure. Sketch the profiling loop and the before/after comparison yourself before checking Advanced.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
critic keeps every rejection reason, not just the last one:
    state["rejection_history"] = state["rejection_history"] + [reason]

final report, once the limit is hit:
    "couldn't converge after N attempts. rejection reasons: [...]"
    -- a reader can see whether the reason repeated (writer is stuck)
    or varied (criteria may be contradictory or genuinely unsatisfiable)

cost-cutting, proven safe:
    fixed test set of 3-5 inputs
    run all 5 through the pipeline BEFORE the cut -> save outputs + total cost
    apply ONE cut to the most expensive stage
    run all 5 through the pipeline AFTER the cut -> save outputs + total cost
    compare: percent cost saved, and whether each of the 5 outputs still
    looks materially the same (not byte-identical, but not degraded)
```

Here's almost the whole thing for Part 1 — fill in the missing piece yourself:

```
# convergence_and_cost_cutting_practice.py
class LoopState(TypedDict):
    draft: str
    revision_count: int
    converged: bool
    rejection_history: list

def critic(state: LoopState) -> Command:
    passed = False
    reason = "draft does not meet required standard (criteria cannot be satisfied"
    history = state["rejection_history"] + [reason]

    if passed:
        return Command(update={"converged": True}, goto=END)

    new_count = state["revision_count"] + 1
    if new_count >= MAX_REVISIONS:
        # your turn: build the final report string using `history`,
        # noting whether the reason repeated every time or varied
        ...
```

```
return Command(update={"revision_count": new_count, "converged": False, "  
return Command(update={"revision_count": new_count, "rejection_history": hist
```

For Part 2, fill in a `compare_before_after(test_inputs, cut_fn)` function yourself: run the test set through both versions, print the cost delta as a percentage, and print each pair of outputs side by side so you can eyeball whether quality held. Then compare both of your finished pieces against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the same two-part shape (a hard-limited loop, and a measured cost cut) at 3 completeness levels — Basic sketches both as pseudocode and confirms nothing runs until invoked, Intermediate is a complete, working non-converging loop with a real limit and the profile-then-cut mechanics, and Advanced adds the diagnostic detail (full rejection history, not just the count) and the proof step (a real before/after test-set comparison, not a one-off measurement) that make both parts' final reports something a reader can actually trust and act on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)